Typescript

What is Typescript?

TypeScript is a programming language developed by Microsoft, built on top of JavaScript. It is an extended version of JavaScript that adds static typing.

Relationship with JavaScript:

- TypeScript is a "superset" of JavaScript, meaning all valid JavaScript code is also valid TypeScript code
- TypeScript is compiled into JavaScript to run in browsers or Node.js
- TypeScript maintains all JavaScript runtime behaviors

Install

```
// install global
npm install -g typescript
```

```
// install into project
npm install typescript --save-dev
```

Type Annotations

Type Annotations in TypeScript are a way to specify data types for variables, function parameters, function return values, and other components in code. This is one of the core features of TypeScript.
Type Annotations make your TypeScript code safer, more readable, and easier to maintain. They allow TypeScript to detect errors early in the development process and provide useful information in the IDE.

Basic Type

Number	String	Boolean
Null	Undefined	Array
Enum	Map(Tuple)	Any

```ts
// Boolean
let isDone: boolean = false;

// Number
let decimal: number = 6;
let hex: number = 0xf00d;
let binary: number = 0b1010;

// String
let color: string = "blue";
let fullName: string = `John ${lastName}`;
// Array
let list: number[] = [1, 2, 3];
let list2: Array<number> = [1, 2, 3]; // Generic syntax
```

```ts
// Tuple
let tuple: [string, number] = ["hello", 10];
map: Map<string, number> = new Map();

// Enum
enum Color {Red, Green, Blue}
let c: Color = Color.Green;

// Any
let notSure: any = 4;
notSure = "maybe a string instead";

// Void
function warnUser(): void {
    console.log("This is a warning message");
}

// Null and Undefined
let u: undefined = undefined;
let n: null = null;
```

Union Types and Intersection Types

```
// Union Type: Variable can be one of the listed types
let value: string | number;
value = “hello"; //OK
value = 42; //OK
value = true; // Error
```

```
// Intersection Type: Combines properties of multiple types
interface A {
    a: number;
}
interface B {
    b: string;
}
type C = A & B;
let c: C = { a: 1, b: "hello" };
```

Generic Type

```
// T is generic type, can be any type
function identity<T>(arg: T): T {
    return arg;
}

// Can specify concrete type
let output = identity<string>(“myString”);

// Or let TypeScript infer
let output2 = identity(42);
// TypeScript infers T is number

// Multiple generic params
function pair<T, U>(first: T, second: U): [T, U] {
    return [first, second];
}

let result = pair<string, number>("hello", 42);
```

```
class Box<T> {
    private value: T;

    constructor(value: T) {
        this.value = value;
    }

    getValue(): T {
        return this.value;
    }
}

let stringBox = new Box<string>("hello");
let numberBox = new Box<number>(42);
```

Generic Type

```
//Generic constraints
interface Lengthwise {
    length: number;
}

function loggingIdentity<T extends Lengthwise>(arg: T): T {
    console.log(arg.length);
    return arg;
}

// OK
loggingIdentity("hello");  // length property of the String prototype
loggingIdentity([1, 2, 3]); // length property of the Array prototype

// Error
loggingIdentity(42);  // number don’t have property length
```

Utility Type

- **Partial<T>**: Makes all properties optional
- **Required<T>**: Makes all properties required
- **Readonly<T>**: Makes all properties readonly
- **Pick<T, K>**: Selects some properties from a type
- **Omit<T, K>**: Removes some properties from a type
- **Record<K, T>**: Creates an object type with specified key and value types
- **Exclude<T, U>**: Removes types from a union type
- **Extract<T, U>**: Extracts common types from two union types
- **NonNullable<T>**: Removes null and undefined from a type
- **ReturnType<T>**: Gets the return type of a function type
- **Parameters<T>**: Gets the types of a function's parameters

```
interface Todo {
    title: string;
    description: string;
    completed: boolean;
}

// Partial: makes all properties optional
type PartialTodo = Partial<Todo>;

// Pick: takes only specified properties
type TodoPreview = Pick<Todo, "title" | "completed">;

// Omit: removes specified properties
type TodoInfo = Omit<Todo, "completed">;
```

Type Guard

Type Guards are techniques that help TypeScript understand the exact data type of a variable within a specific code scope.

```
function isString(value: any): value is string {
    return typeof value === "string";
}

function example(x: string | number) {
    if (isString(x)) {
        // x is string here
        console.log(x.toUpperCase());
    } else {
        // x is number here
        console.log(x.toFixed(2));
    }
}
```

Type Guard

typeof Type Guards

```
function isString(value: any): value is string {
    return typeof value === "string";
}

function example(x: string | number) {
    if (isString(x)) {
        // x is string here
        console.log(x.toUpperCase());
    } else {
        // x is number here
        console.log(x.toFixed(2));
    }
}
```

Type Guard

instanceof Type Guards

```
class Animal {
    name: string;
    constructor(name: string) {
        this.name = name;
    }
}

class Dog extends Animal {
    bark() {
        console.log("Woof!");
    }
}

function processAnimal(animal: Animal) {
    if (animal instanceof Dog) {
        // In this block, TypeScript knows animal is Dog
        animal.bark();
    } else {
        // In this block, TypeScript knows animal is Animal
        console.log(animal.name);
    }
}
```

Type Guard

In Type Guards

```ts
interface Bird {
    fly(): void;
}

interface Fish {
    swim(): void;
}

function move(pet: Bird | Fish) {
    if ("fly" in pet) {
        // In this block, TypeScript knows pet is Bird
        pet.fly();
    } else {
        // In this block, TypeScript knows pet is Fish
        pet.swim();
    }
}
```

Type Guard

User-Defined Type Guards

```
// Type predicate
function isString(value: any): value is string {
    return typeof value === "string";
}

function isNumber(value: any): value is number {
    return typeof value === "number";
}

function processValue(value: string | number) {
    if (isString(value)) {
        // TypeScript knows value is string
        console.log(value.toUpperCase());
    } else if (isNumber(value)) {
        // TypeScript knows value is number
        console.log(value.toFixed(2));
    }
}
```

Function Type

In TypeScript, a "function type" is a data type that represents a function, including both its parameter types and return type. It helps TypeScript determine the structure of a function before it is executed, aiding in early error detection and enhancing code stability.

```
type GreetFunction = (name: string) => string;

const greet: GreetFunction = (name) => {
    return `Hello, ${name}!`;
};
```

Function Type Example

```ts
type numberToString = (a: number, b: number) => string;

function doSomething(a: number, b: number): string {
    return (a + b).toString();
}
const doSomething2: numberToString = (a: number, b: number): string => {
    return (a + b).toString();
}
function doSomething3(a: number, b: number): number {
    return ((a + b) * 2);
}
```

```ts
let myFunc: numberToString;

myFunc = doSomething;
console.log(myFunc(1, 2));

myFunc = doSomething2;
console.log(myFunc(1, 2));

myFunc = doSomething3;
console.log(myFunc(1, 2));
```

Type Assertion

In TypeScript, "Type Assertion" is a way to tell the compiler to treat a value as a specific type, even if the compiler cannot infer that type automatically. It's like telling TypeScript "trust me, I know what I'm doing" with the type of a value. Syntax is : as or <>

```
let someValue: any = "this is a string";
let strLength: number = (someValue as string).length;
let strLength2: number = (<string>someValue).length;
```

```typescript
interface User {
    id: number;
    name: string;
    age: number;
}

async function fetchUser(): Promise<User> {
    const response = await fetch(‘…api.example.com/user');
    const data = await response.json();
    // Type Assertion because we trust the API returns
correct format
    return data as User;
}

function processFormData(formData: unknown): User {
    if (typeof formData === 'object' && formData !== null)
{
        const userData = formData as Partial<User>;
        if (!userData.name || !userData.age) {
            throw new Error('Invalid user data');
        }
        return {
            id: userData.id || 0,
            name: userData.name,
            age: userData.age
        };
    }
    throw new Error('Invalid form data');
}
```

```typescript
async function main() {
    try {
        // Fetch user data
        const user = await fetchUser();
        console.log('User:', user);

        // Process form data
        const formData = { name: 'John', age: 30 };
        const processedUser = processFormData(formData);
        console.log('Processed user:', processedUser);
    } catch (error) {
        console.error('Error:', error);
    }
}

main();
```

Interface ****

```
interface Person {
    name: string;
    age: number;
    email?: string; // Optional property
    readonly id: number; // Readonly property
}

let person: Person = {
    name: "John",
    age: 30,
    id: 1
};
```

Notice

- Typescript can infer type in many case without explicit declaration
- Non-null or Undefined assertion operatior: !